

Trabajo Práctico 2 — Catán

[7507/9502] Algoritmos y Programación III

Curso 01

Segundo cuatrimestre de 2025

Alumno	Padron
Turco Gianfranco	110073
Aldrey Daiana	110624
Amarilla Carolina	111995
Povis Lucía	112211
Rojas Joselín	107170

Índice

1. Introducción	2
2. Detalles de implementación	2
3. Diagramas de clase	4
4. Excepciones	12
5. Diagramas de secuencia	20

1. Introducción

El presente informe reúne la documentación de la solución del segundo trabajo práctico de la materia Algoritmos y Programación III que consiste en desarrollar el juego Catán a través de los métodos y conocimientos adquiridos a lo largo de la materia, utilizando Java como lenguaje de desarrollo y JavaFx para su interfaz gráfica.

2. Detalles de implementación

El tablero utilizado es el mostrado a continuación, el mismo fue construido mediante una clase Grafo con sus correspondientes vértices y aristas. Existen dos tipos de vértices, los VerticeTerreno donde van los terrenos, señalados con letras y los VerticeEdificio donde van las piezas, señalados con números.

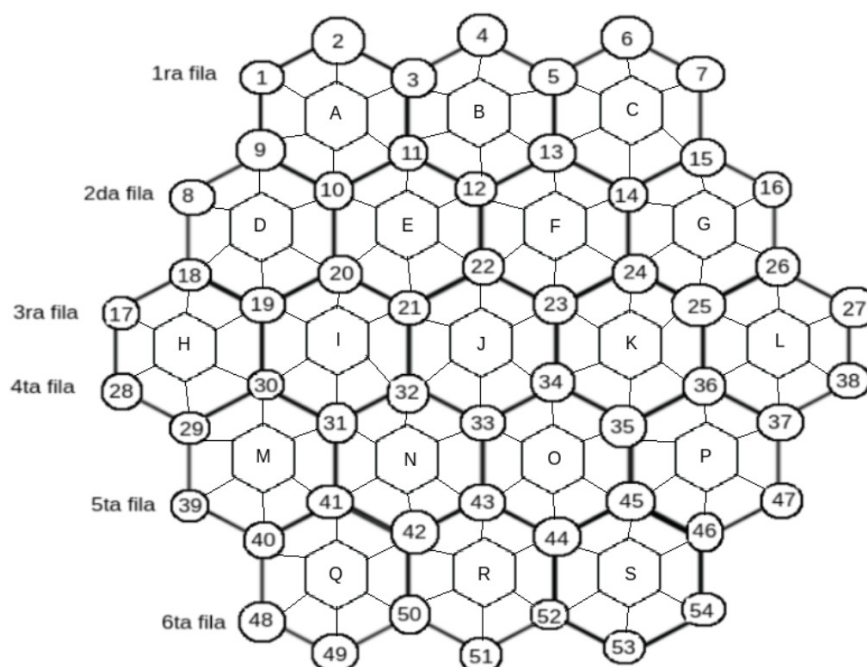


Figura 1: Grafo.

A lo largo del desarrollo del trabajo práctico se intentó poner foco en utilizar algunos patrones de diseño que fueron de vital importancia para su correcta ejecución. Más allá de esto, se pusieron a prueba las herramientas necesarias para mantener en pie todos los fundamentos de la programación orientada a objetos: polimorfismo, utilización de herencias, interfaces y todos los tipos de relaciones, encapsulamiento, alta cohesión y bajo acoplamiento.

- Patron NULL: Ciertas clases abstractas tienen clases hijas llamadas NoClase o ClaseNula, de esta manera se logra inicializar esta misma clase aquellas que las necesiten, sin necesidad de una inicialización NULL. Se respeta el principio de sustitución de Liskov y se evitan chequeos repetitivos. Algunas clases que lo implementan son: Pieza o Recurso.

```
public VerticeEdificio(UbicacionVertice ubicacion) {  
    terrenos = new ArrayList<>();  
    this.ubicacion = ubicacion;  
    disponible = true;  
    pieza = new NoPieza(ubicacion);  
}
```

Figura 2: Ejemplo Patrón NULL.

- Singleton: Se utiliza para garantizar una única instancia de la clase en todo el programa y que la misma sea accesible globalmente. Ejemplos de esto se pueden observar en las clases Tablero o Banco.

```
public final class Tablero {  daiana +2  
    private Grafo grafo; 15 usages  
    private List<Pieza> piezas; 4 usages  
    private List<Terreno> terrenos; 2 usages  
    private Ladron ladron; 4 usages  
    private List<Puerto> puertos; 4 usages  
  
    private static final Tablero INSTANCE = new Tablero(); 1 usage  
  
    private Tablero() { 1 usage  daiana +1  
        this.grafo = new Grafo();  
        this.terrenos = new ArrayList<>();  
        this.piezas = new ArrayList<>();  
        this.ladron = new Ladron();  
        this.puertos = new ArrayList<>();  
  
        crearGrafo();  
    }  
  
    public static Tablero getInstance() { return INSTANCE; }
```

Figura 3: Ejemplo Singleton.

- Double dispatch: Permite que dos objetos participen en la decisión de qué método usar. Útil para evitar el tell dont ask. Se utiliza en Recurso.

```

public class Ladrillo extends Recurso {
    @amarillacarolina
    public Ladrillo() { super(); }

    public Ladrillo(int cant) { super(cant); }

    @Override @amarillacarolina
    public void podesIncrementar(Recurso recursoDelJugador, int cantidad) {
        recursoDelJugador.incrementar(recursorecibido: this, cantidad);
    }

    @Override @amarillacarolina
    protected void incrementar(Madera recursorecibido, int cant) { throw new RecursoIncorrecto(); }

    @Override @amarillacarolina
    protected void incrementar(Mineral recursorecibido, int cant) { throw new RecursoIncorrecto(); }

    @Override @amarillacarolina
    protected void incrementar(Ladrillo recursorecibido, int cant) { this.incrementar(cant); }

    @Override @amarillacarolina
    protected void incrementar(Lana recursorecibido, int cant) { throw new RecursoIncorrecto(); }

    @Override @amarillacarolina
    protected void incrementar(Grano recursorecibido, int cant) { throw new RecursoIncorrecto(); }
}

```

Figura 4: Ejemplo Double Dispatch.

3. Diagramas de clase

A continuación se presentan los diagramas de clase correspondientes al modelo del trabajo.

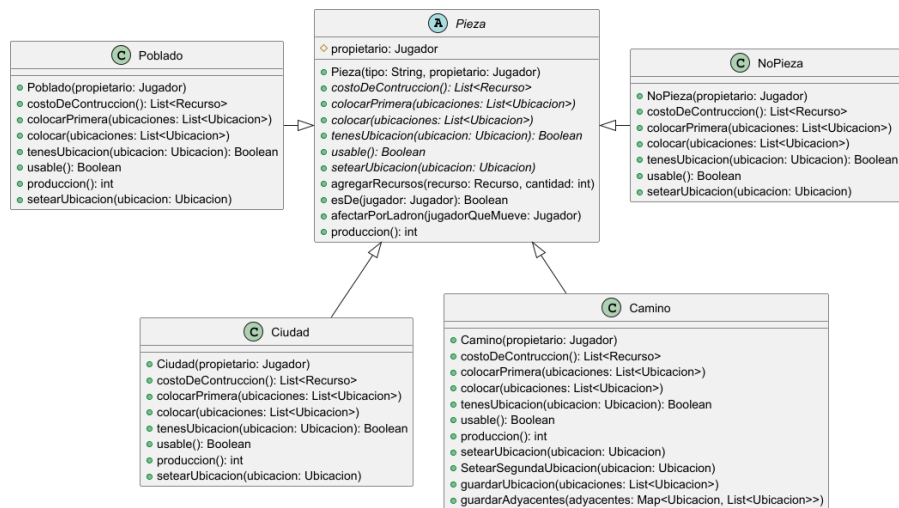


Figura 5: Diagrama de Pieza.

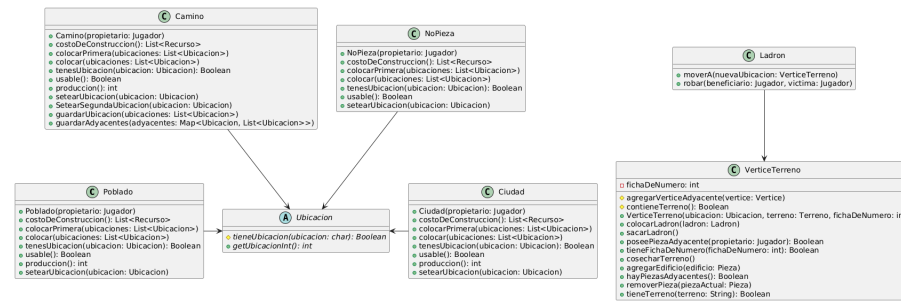


Figura 6: Diagrama de Ladron y Pieza.

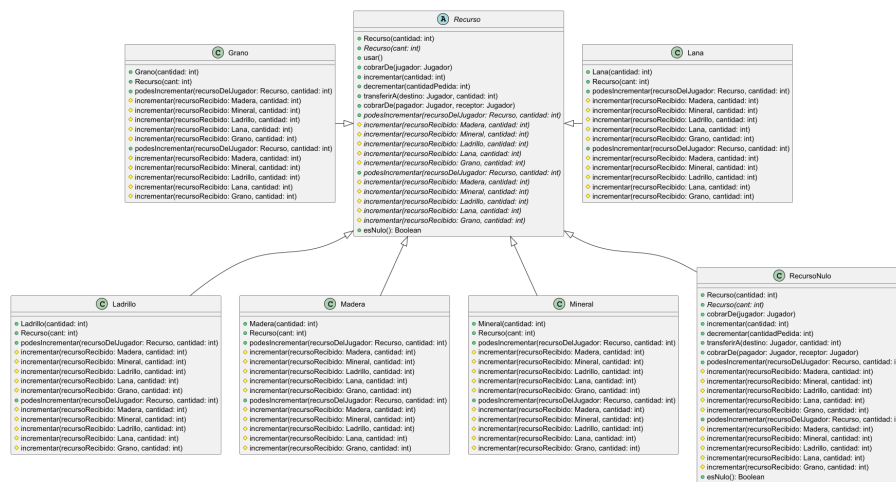


Figura 7: Diagrama de Recurso.

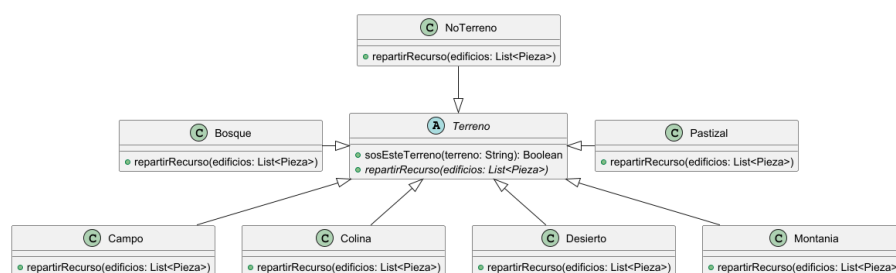


Figura 8: Diagrama de Terreno.

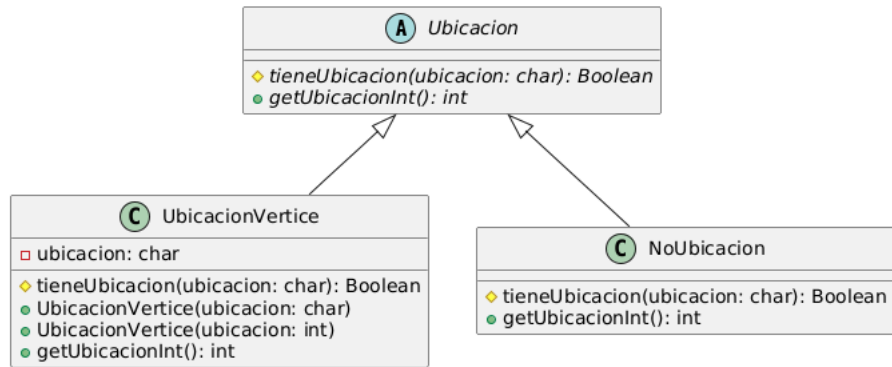


Figura 9: Diagrama de Ubicacion.

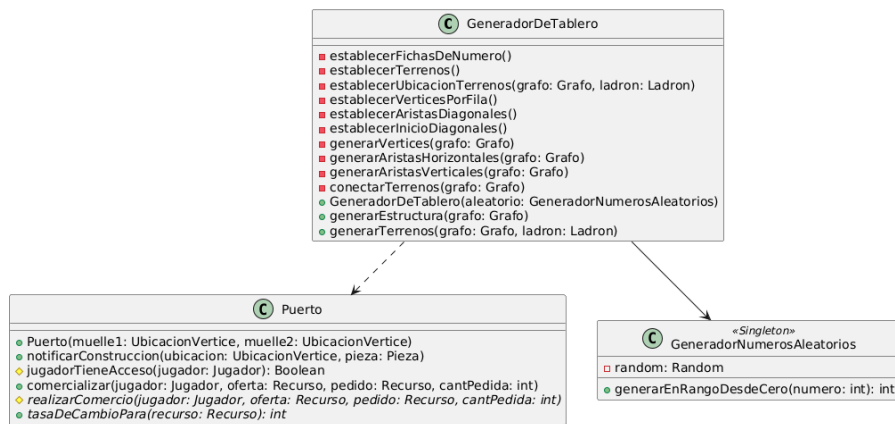


Figura 10: Diagrama de Generadores.

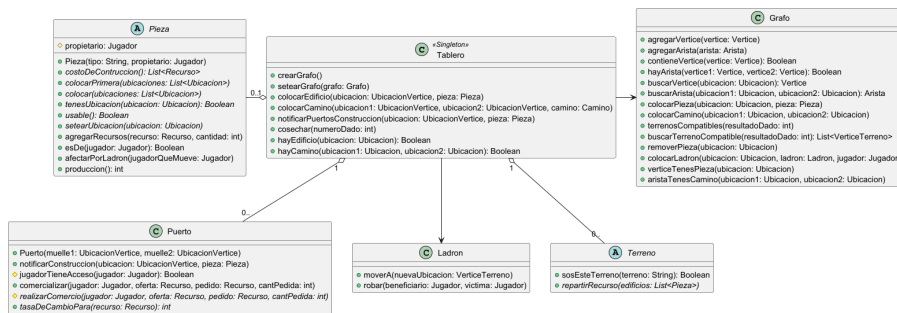


Figura 11: Diagrama de Tablero.



Figura 12: Diagrama de Grafo.

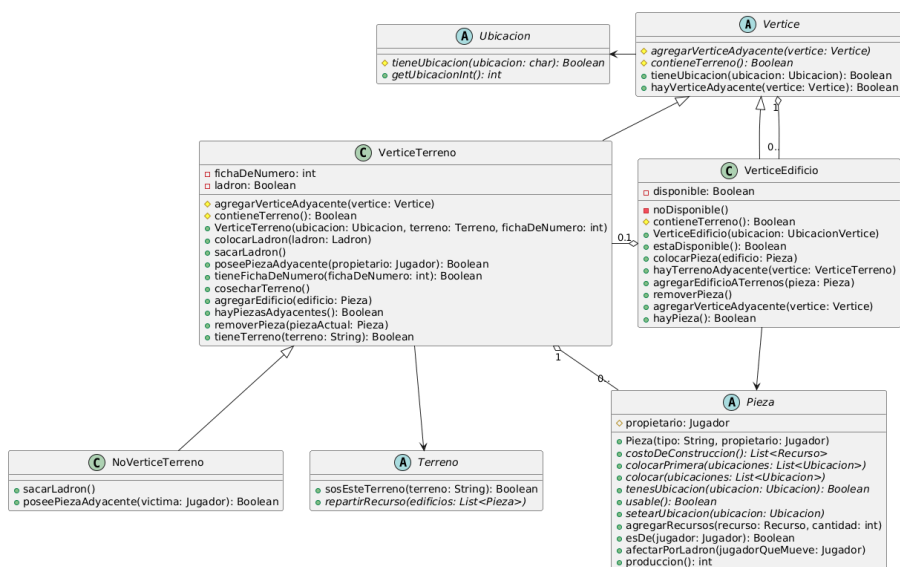


Figura 13: Diagrama de Vertice.

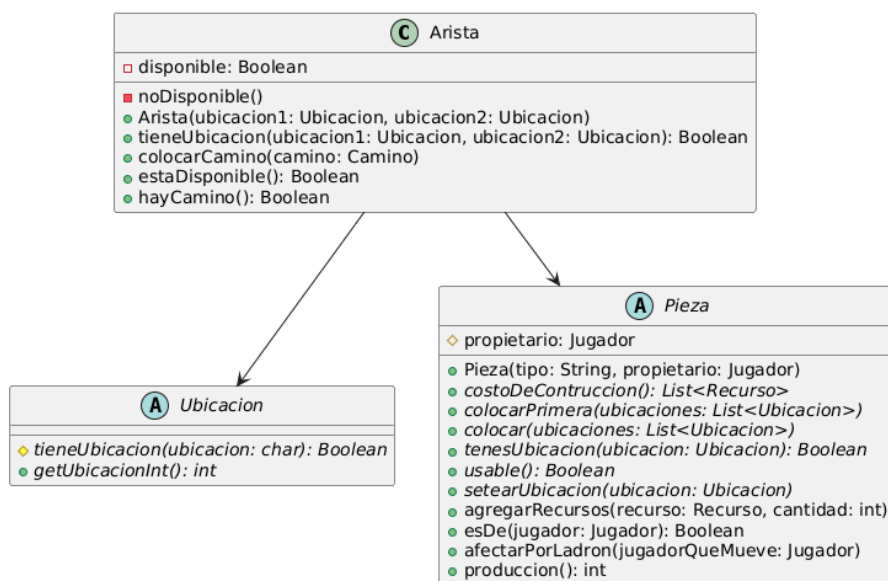


Figura 14: Diagrama de Arista.

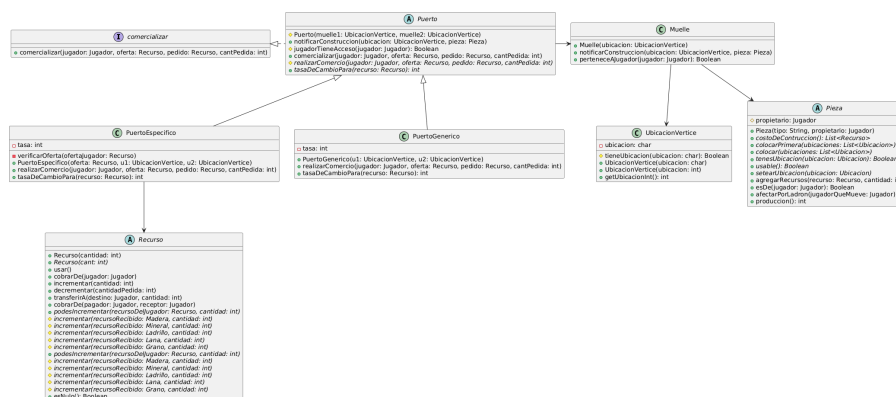


Figura 15: Diagrama de Puerto.

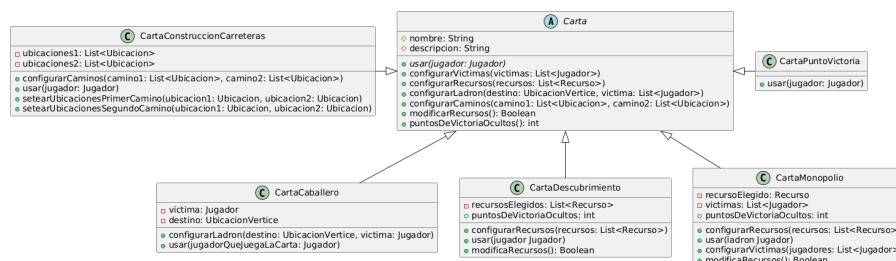


Figura 16: Diagrama de Carta.

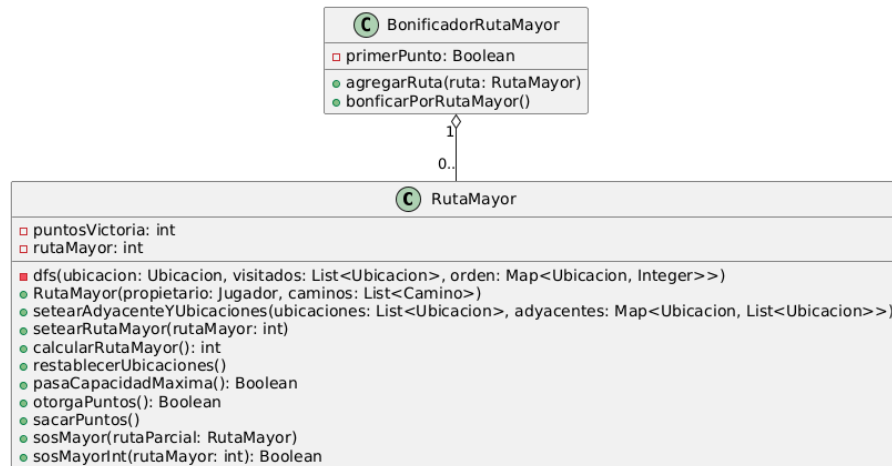


Figura 17: Diagrama de BonificadorRutaMayor.

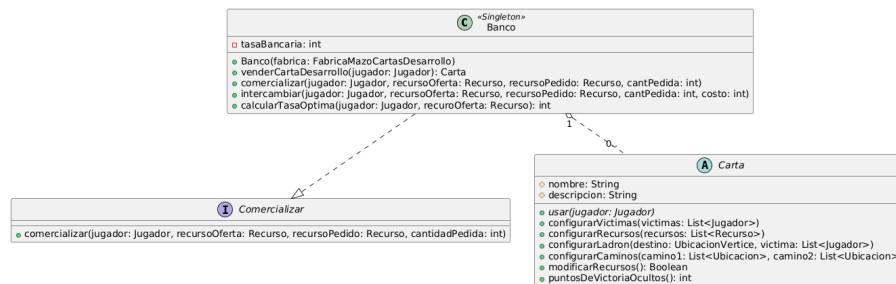


Figura 18: Diagrama de Banco.

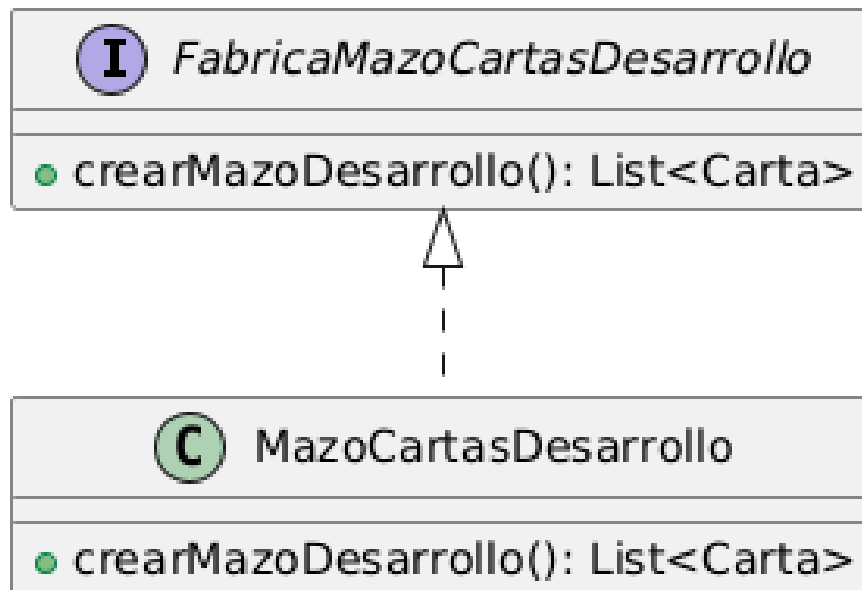


Figura 19: Diagrama de Mazo.

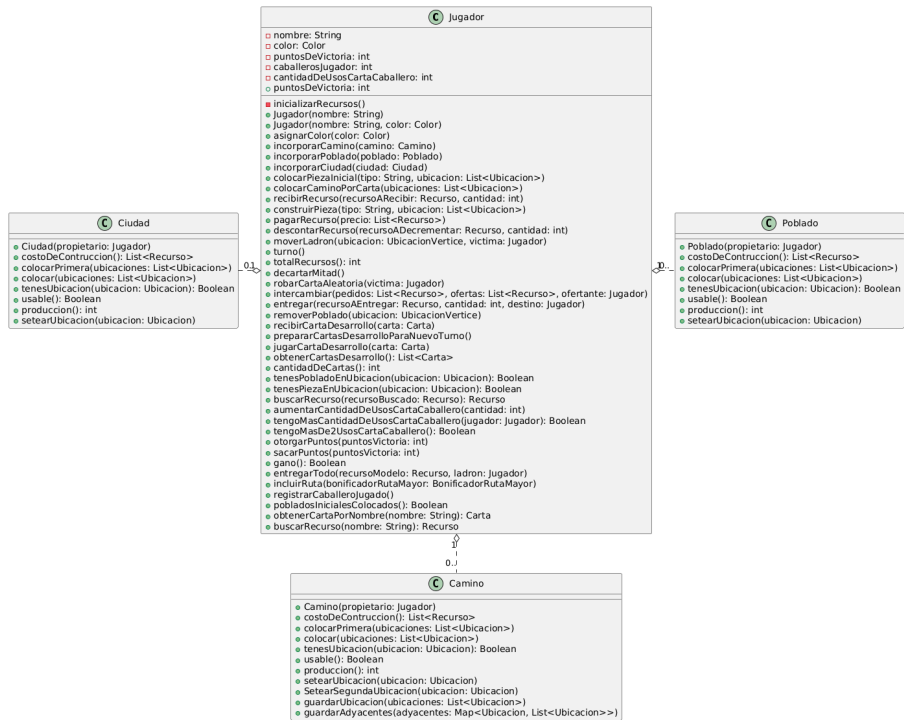


Figura 20: Diagrama de Jugador parte 1.



Figura 21: Diagrama de Jugador parte 2.



Figura 22: Diagrama de Jugador parte 3.

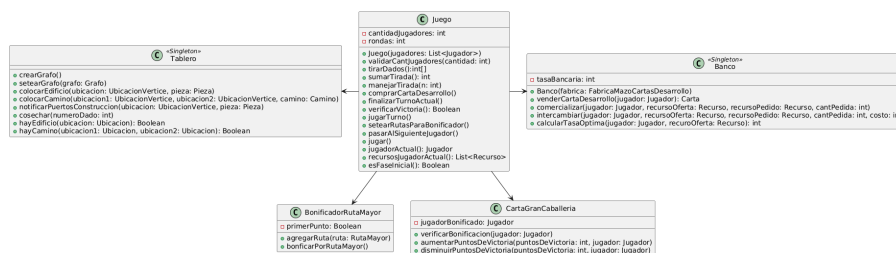


Figura 23: Diagrama de Juego parte 1.



Figura 24: Diagrama de Juego parte 2.

4. Excepciones

AccionNoPermitida Se lanza en la clase Carta al realizar acciones que no estan permitidas por la carta seleccionada.

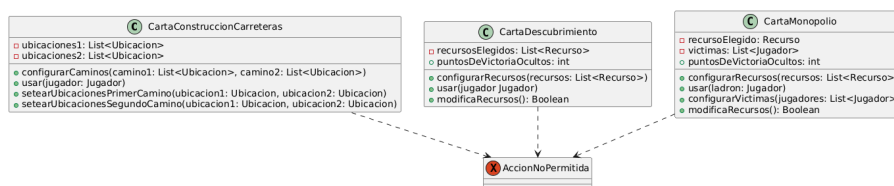


Figura 25: AccionNoPermitida.

AristaInvalida Se lanza en la clase Grafo al querer agregar una arista ya existente o al querer buscar una arista inexistente.

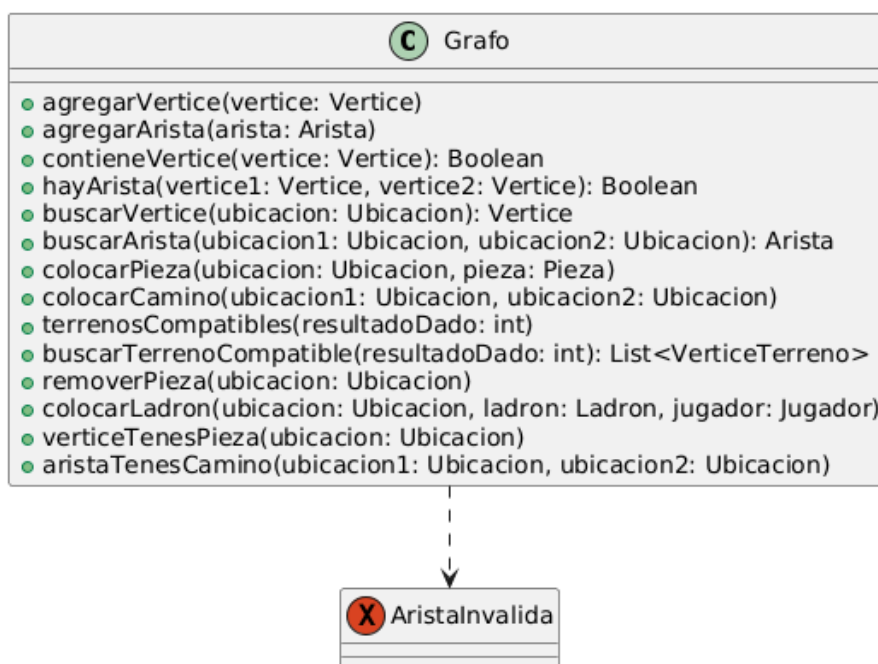


Figura 26: AristaInvalida.

CantidadUbicacionesInvalida Se lanza en las piezas al pasar por parametro una cantidad inválida de ubicaciones ya sea para poblado o para camino.

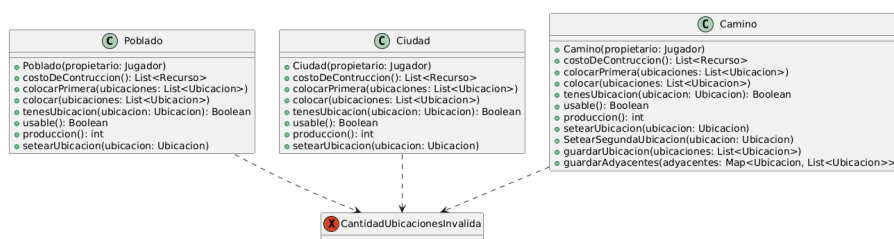


Figura 27: CantidadUbicacionesInvalida.

ErrorNoUsoDeCartaInvalido Se lanza al intentar utilizar una carta justo después de su adquisición.



Figura 28: ErrorNoUsoDeCartaInvalido.

CantJugadoresInvalida Se lanza cuando se intenta crear una cantidad de jugadores incorrecta.

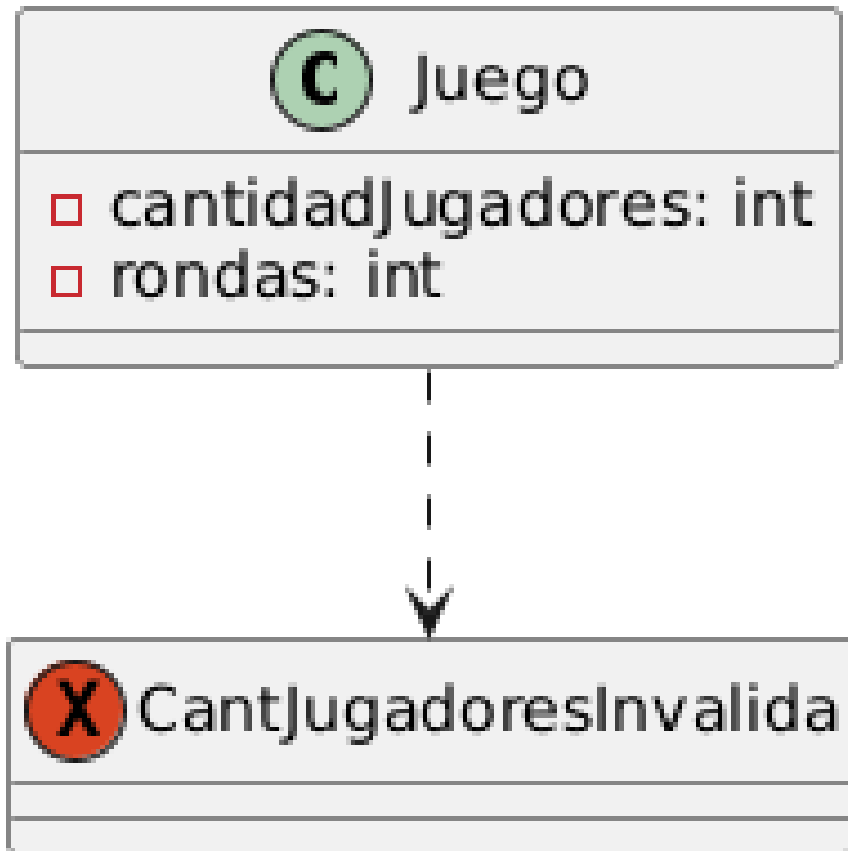


Figura 29: CantJugadoresInvalida.

NoTieneCarta Se lanza cuando no se tienen mas cartas, ya sea el jugador o el banco.

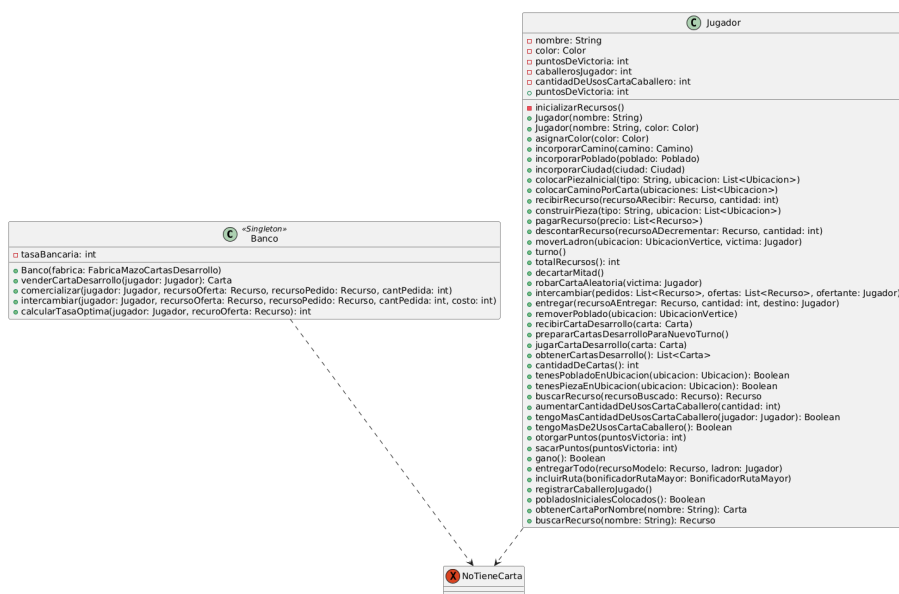


Figura 30: NoTieneCarta.

ObjetoNoUsable Se lanza al intentar utilizar los comportamientos de los NoObjetos como NoPieza o NoTerreno.

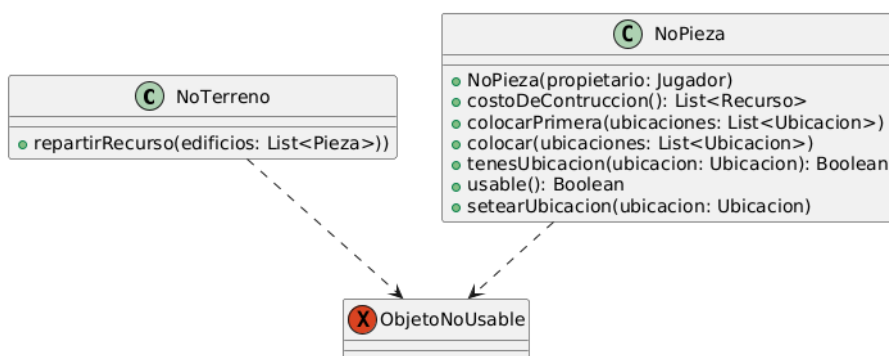


Figura 31: ObjetoNoUsable.

PiezaNoEncontrada Se lanza cuando se intenta crear una pieza que no corresponde al juego.

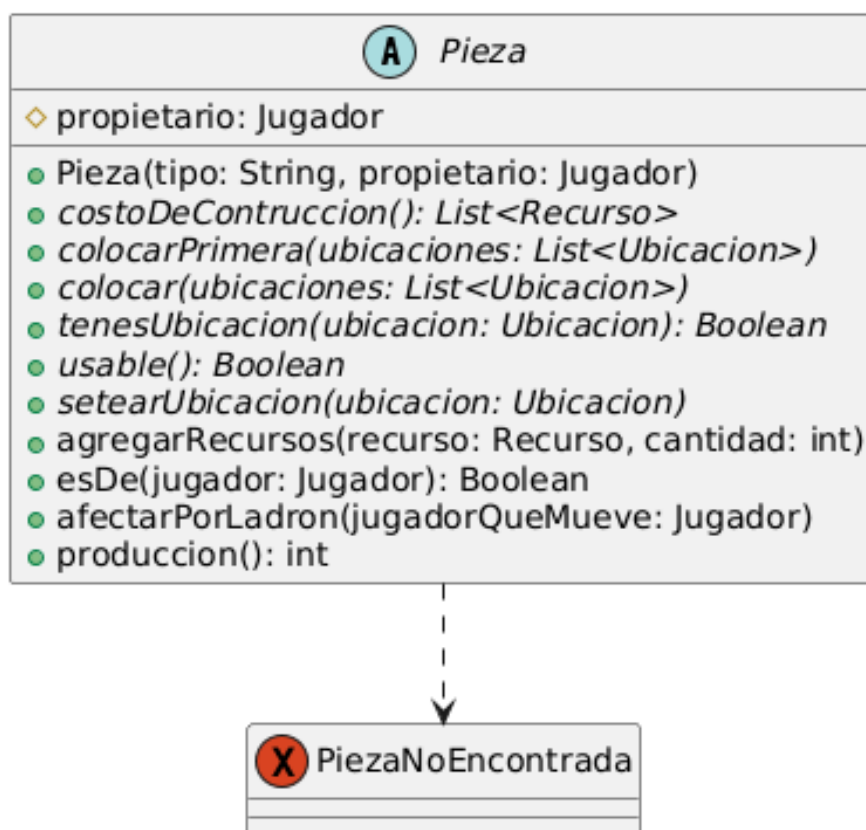


Figura 32: PiezaNoEncontrada.

PuertoNoAccesible Se lanza cuando el jugador intenta acceder a un puerto que no está permitido.

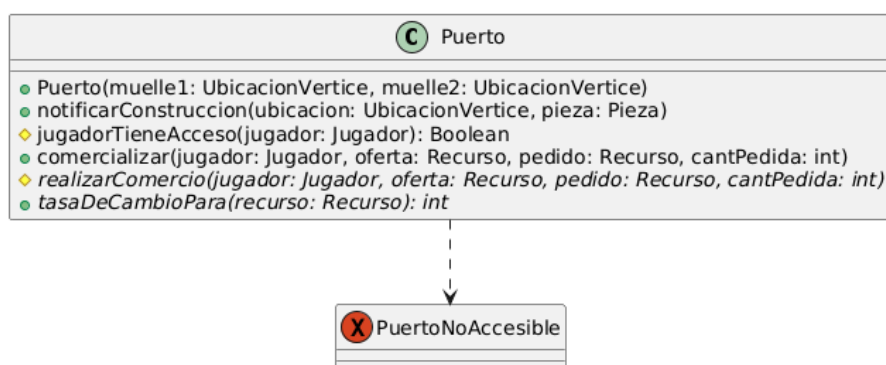


Figura 33: PuertoNoAccesible.

RecursoIncorrecto Se lanza al no coincidir los recursos en el double dispatch dentro de la propia clase.

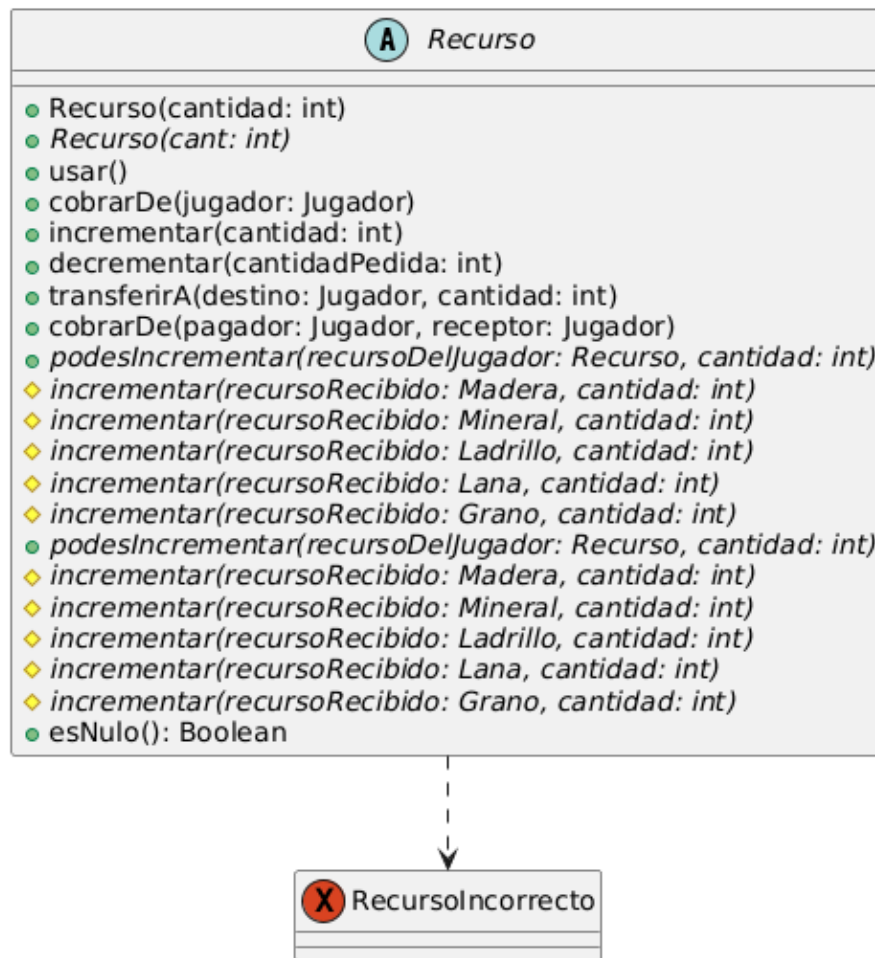


Figura 34: RecursoIncorrecto.

RecursoOfertaIncorrecto Se lanza al intentar ofrecer una cantidad errónea de recursos en un puerto.

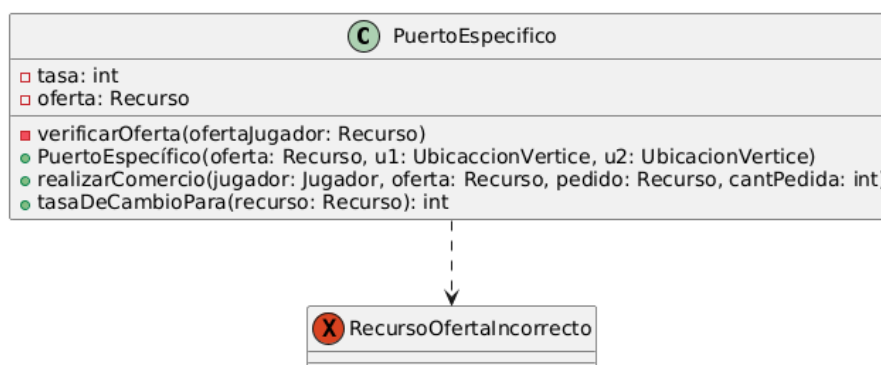


Figura 35: RecursoOfertaIncorrecto.

SinPiezas Se lanza cuando un VerticeTerreno no tiene piezas adyacentes para entregar recursos.

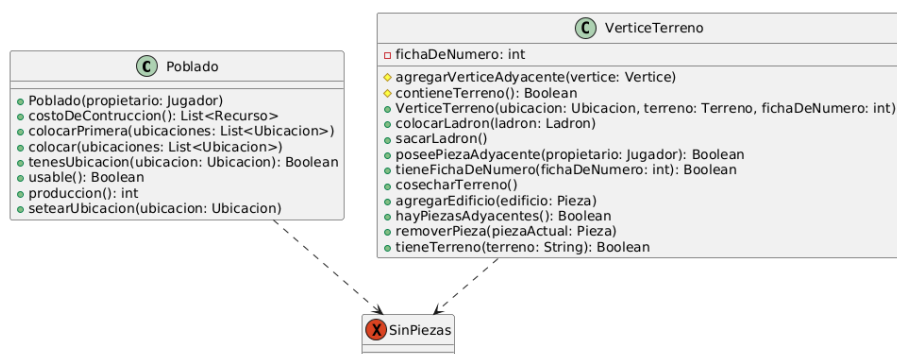


Figura 36: SinPiezas.

SinRecursos Se lanza cuando se intenta realizar acciones con un recurso que no hay o cuando no existen las cantidades suficientes en un intercambio.

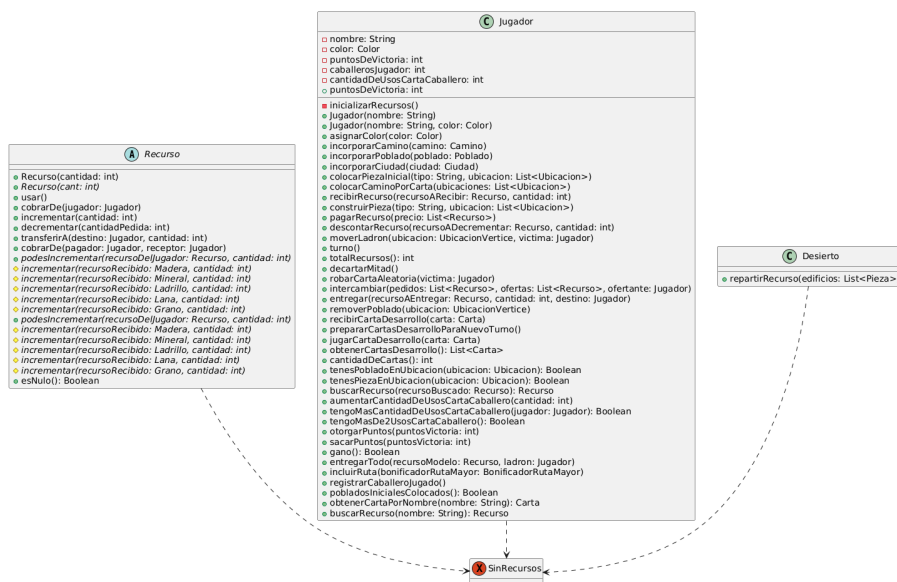


Figura 37: SinRecursos.

VerticeInvalido Se lanza cuando se intenta buscar un vértice que no fue creado o cuando se coloca/remueve una pieza de un vertice que no corresponde.

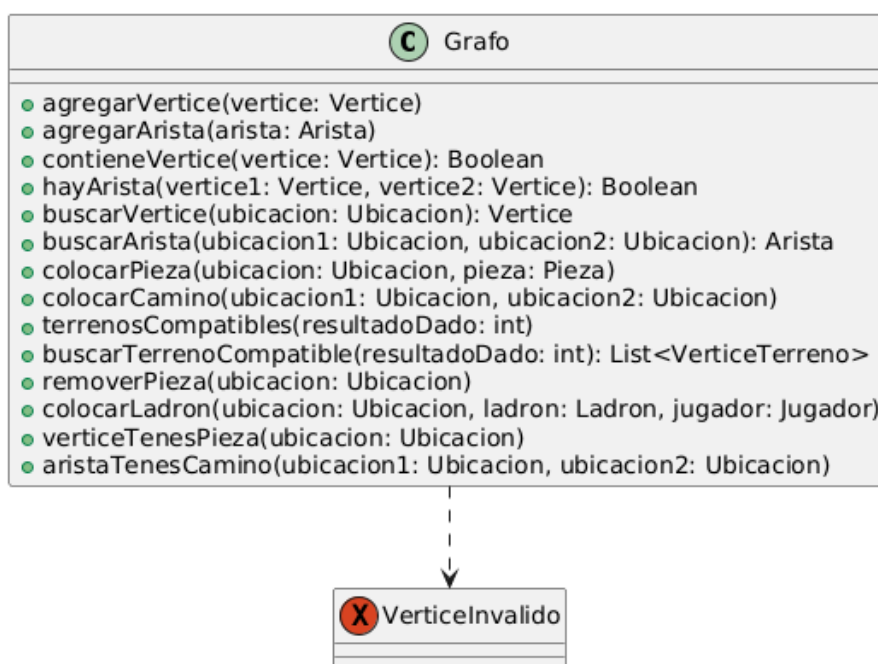


Figura 38: VerticeInvalido.

VictimaInválida Se lanza cuando el jugador al que robarle no posee piezas adyacentes.

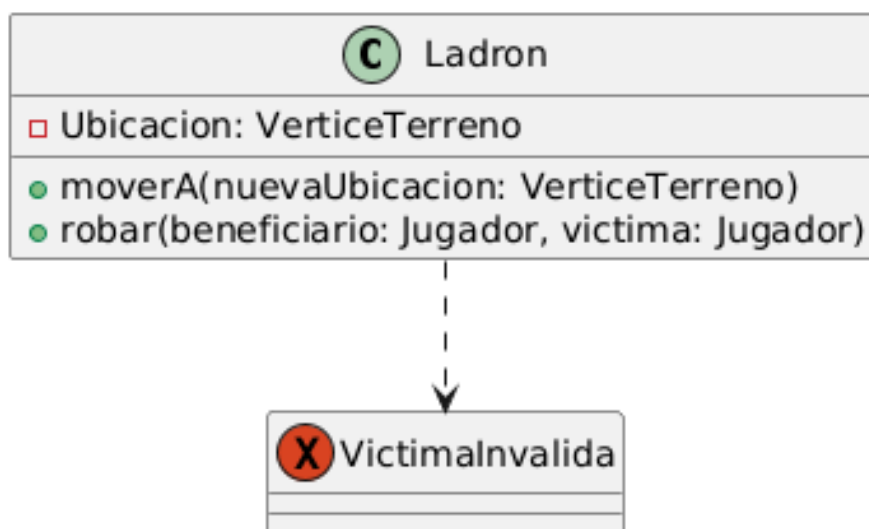


Figura 39: VictimaInvalida.

5. Diagramas de secuencia

El diagrama de secuencia seleccionado corresponde a uno de los test integrales de la entrega número 2, en donde se observa el recorrido de clases que se realiza a la hora de que un jugador coloque una pieza en el tablero.

@Test validacionDelConsumoDeRecursosYLaCorrectaColocacionDeUnaCarretera()

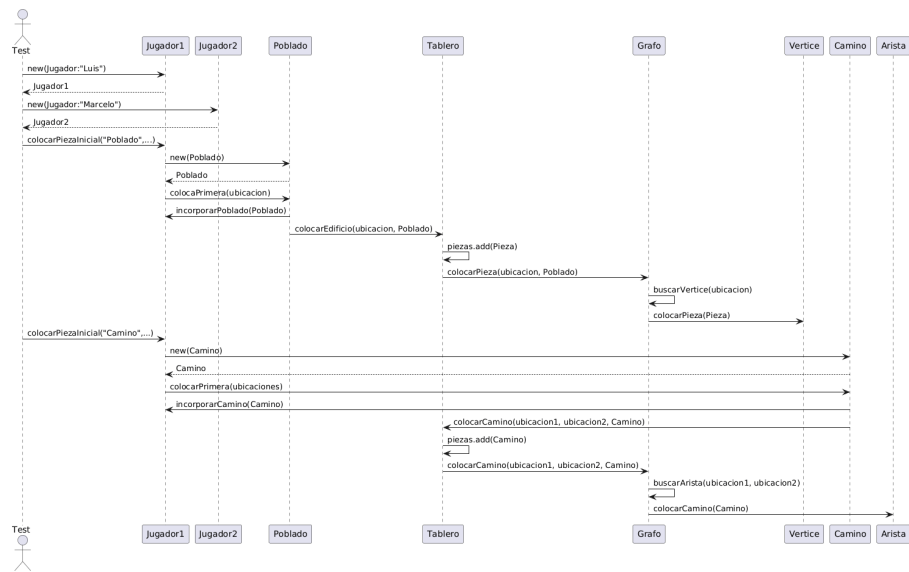


Figura 40: Parte 1.

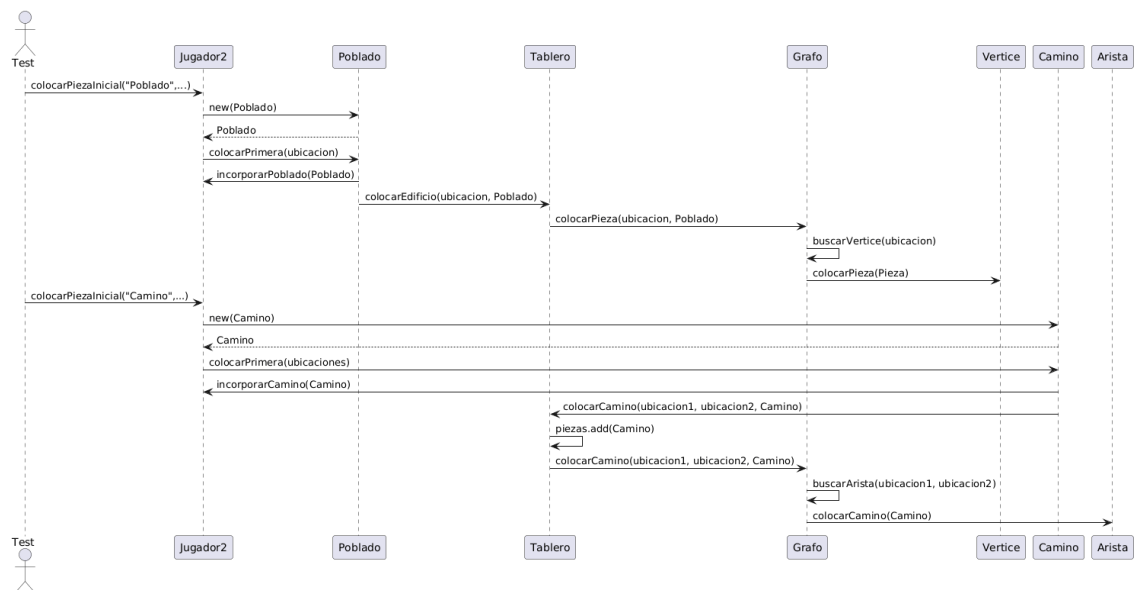


Figura 41: Parte 2.

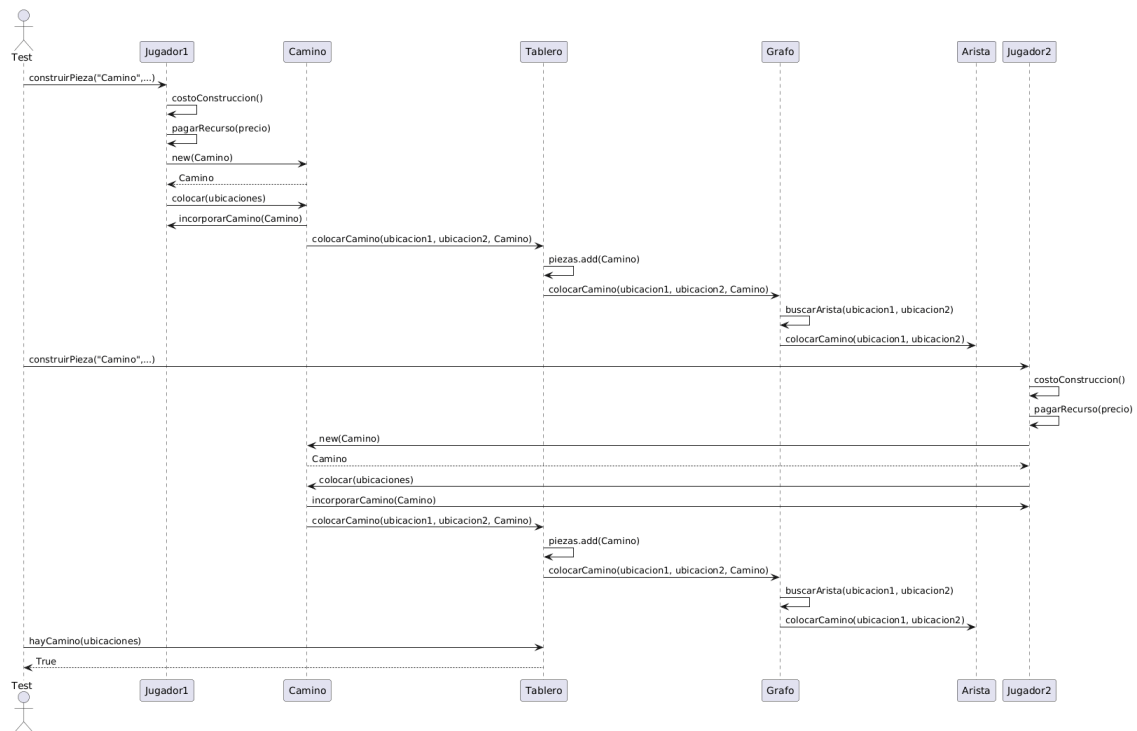


Figura 42: Parte 3.